Implement Dijsktra's algorithm to compute the shortest path through a network

Aim: To write C Program to implement Dijkstra's algorithm to find shortest path

Objective:

Dijkstra's Algorithm allows you to calculate the shortest path between one node (you pick which one) and *every other node in the graph.* Dijkstra algorithm is also called single source shortest path algorithm. It is based on greedy technique.

Dijkstra's Algorithm:

The algorithm maintains a list visited[ ] of vertices, whose shortest distance from the source is already known. If visited[1], equals 1, then the shortest distance of vertex i is already known. Initially, visited[i] is marked as, for source vertex.

At each step, we mark visited[v] as 1. Vertex v is a vertex at shortest distance from the source vertex. At each step of the algorithm, shortest distance of each vertex is stored in an array distance[ ].

Step 1 :

Create cost matrix C[ ][ ] from adjacency matrix adj[ ][ ].

C[i][j] is the cost of going from vertex i to vertex j.

If there is no edge between vertices i and j then C[i][j] is infinity.

Step 2 : Array visited[ ] is initialized to zero.

```
for(i=0;i<n;i++)
        visited[i]=0;
```

Step 3: If the vertex 0 is the source vertex then visited[0] is marked as 1.

Step 4:

1. Create the distance matrix, by storing the cost of vertices from vertex no. 0 to n-1 from the source vertex 0.

```
for(i=1;i<n;i++)
        distance[i]=cost[0][i];
```

2. Initially, distance of source vertex is taken as 0. i.e. distance[0]=0;

Step 5 : for(i=1;i<n;i++)

1. Choose a vertex w, such that distance[w] is minimum and visited[w] is 0. Mark visited[w] as
2. Recalculate the shortest distance of remaining vertices from the source.
3. Only, the vertices not marked as 1 in array visited[ ] should be considered for recalculation of distance. i.e. for each vertex v

```
    if(visited[v]==0)
```

distance[v]=min(distance[v],distance[w]+cost[w][v])

C Program:

```
#include<stdio.h>
#define INFINITY 9999
#define MAX 10
void dijkstra(int G[MAX][MAX],int n,int startnode);
int main()
{
	int G[MAX][MAX],i,j,n,u;
	printf("Enter no. of vertices:");
	scanf("%d",&n);
	printf("\nEnter the adjacency matrix:\n");
	for(i=0;i<n;i++)
		for(j=0;j<n;j++)
			scanf("%d",&G[i][j]);
	printf("\nEnter the starting node:");
	scanf("%d",&u);
	dijkstra(G,n,u-1);
	return 0;
}
 void dijkstra(int G[MAX][MAX],int n,int startnode)
{
	int cost[MAX][MAX],distance[MAX],pred[MAX];
	int visited[MAX],count,mindistance,nextnode,i,j;
	//pred[] stores the predecessor of each node
	//count gives the number of nodes seen so far
	//create the cost matrix
	for(i=0;i<n;i++)
		for(j=0;j<n;j++)
			if(G[i][j]==0)
				cost[i][j]=INFINITY;
```

```
                else
                        cost[i][j]=G[i][j];
        //initialize pred[ ],distance[ ] and visited[ ]
        for(i=0;i<n;i++)
        {
                distance[i]=cost[startnode][i];
                pred[i]=startnode;
                visited[i]=0;
        }
         distance[startnode]=0;
        visited[startnode]=1;
        count=1;
        while(count<n-1)
        {
                mindistance=INFINITY;
                //nextnode gives the node at minimum distance
                for(i=0;i<n;i++)
                        if(distance[i]<mindistance&&!visited[i])
                        {
                                mindistance=distance[i];
                                nextnode=i;
                        }
                        //check if a better path exists through nextnode
visited[nextnode]=1;
                        for(i=0;i<n;i++)
                                if(!visited[i])
                                if(mindistance+cost[nextnode][i]<distance[i])
                                        {
                                distance[i]=mindistance+cost[nextnode][i];
                                                pred[i]=nextnode;
                                        }
                count++;
```

```
        }

        //print the path and distance of each node
        for(i=0;i<n;i++)
        if(i!=startnode)
        {
        printf("\nTotal Cost from node %d  to %d is %d",startnode+1, i+1,distance[i]);
         printf("\nPath : %d",i+1);
        j=i;
                        do
                        {
                                j=pred[j];
                                printf(" <-- %d",j+1);
                        }while(j!=startnode);
        }
}
```

OUTPUT:

Enter no. of vertices:6

Enter the adjacency matrix:

0 7 9 0 0 14

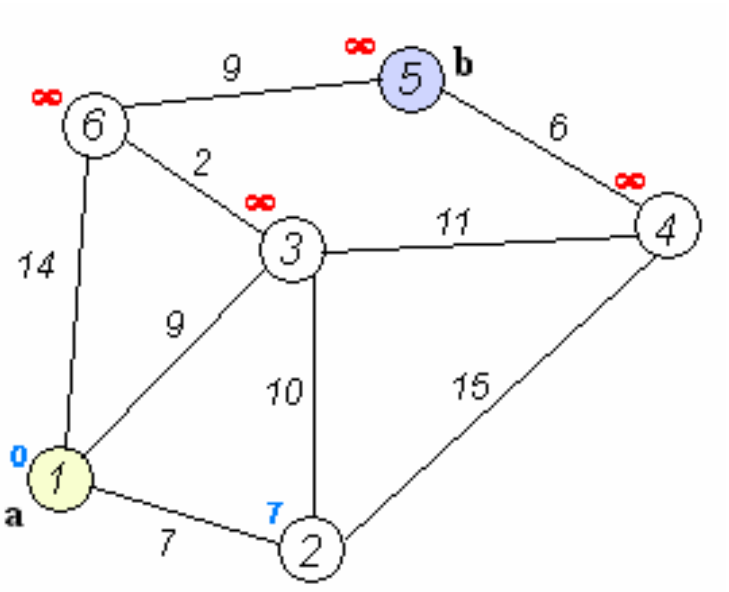
7 0 10 15 0 0

9 10 0 11 0 2

0 15 11 0 6 0

0 0 0 6 0 9

4 0 2 0 9 0



Enter the starting node:1

Total Cost from node 1 to 2 is 7

Path : 2 <-- 1

Total Cost from node 1 to 3 is 9

Path : 3 <-- 1

Total Cost from node 1 to 4 is 20

Path : 4 <-- 3 <-- 1

Total Cost from node 1 to 5 is 20

Path : 5 <-- 6 <-- 3 <-- 1

Total Cost from node 1 to 6 is 11

Path : 6 <-- 3 <-- 1

Enter the starting node: 3

Total Cost from node 3 to 1 is 6

Path : 1 <-- 6 <-- 3

Total Cost from node 3 to 2 is 10

Path : 2 <-- 3

Total Cost from node 3 to 4 is 11

Path : 4 <-- 3

Total Cost from node 3 to 5 is 11

Path : 5 <-- 6 <-- 3

Total Cost from node 3 to 6 is 2

Path : 6 <-- 3

Enter the starting node:4

Total Cost from node 4 to 1 is 17

Path : 1 <-- 6 <-- 3 <-- 4

Total Cost from node 4 to 2 is 15

Path : 2 <-- 4

Total Cost from node 4 to 3 is 11

Path : 3 <-- 4

Total Cost from node 4 to 5 is 6

Path : 5 <-- 4

Total Cost from node 4 to 6 is 13

Path : 6 <-- 3 <-- 4